Feasibility Analysis and Requirements Specification

Smart University Website

Group S | Wai Yan Lynn (67111411) · Pyae Sone (67111336) · Nang Ying Noom Lao (67111378) · Myat Hsu Hnin (68111423)
IT Project I and II (BC362916 + BC362914) | Instructor: Dr. Aftab Alam Janisar | Version 3, revised 17 October 2026
This version replaces version 2 of 2 October 2026. Appendix A lists every change. The target institution is a fictional university, Thales University, referred to as the University. No real institutional data is used.

Section 1: Problem Summary and Target Domain

1.1 Problem Restatement

Universities have already put individual services online. Our review of 17 universities in Section 1.2 shows that the problem is not a missing service. It is that services are split apart, one-way, and cannot be reached by conversation. Three problems follow.

- **Services are split across separate systems.** A student who needs a transcript, a study room and a library book uses three different systems, each with its own address and login, and has to know which system does which job. We found an online document request route at 13 of the 17 universities, but in every case it sat in a separate registrar system, away from the main website and away from the room and library systems.
- **Information is published but hard to find.** Rules and fee information sit inside PDF handbooks and old announcement posts, and site search matches page titles rather than the rule inside the file. A student has to know which document holds the answer before they can find it, so many ask a senior instead, and unofficial answers spread faster than official ones.
- **Case records exist, but they are hard to share.** Each staff member keeps records in a personal inbox or spreadsheet instead of one shared system. If the person who handled a case is away the next day, a student collecting visa documents has to explain everything again, and the next staff member repeats the same search. The student cannot see the state of the request either: of the 17 universities, only 3 showed request status to the student, and none of the 12 Thai universities did.

Why the assistants that already exist do not close this gap. Fifteen of the 17 universities offer an assistant or an official chat channel, including the Chulalongkorn registrar LINE account, Bangkok University's student question channel, ASU's Sunny and several at NUS. Thai universities have also published intent-based LINE chatbots, such as NN Bot at KMUTNB and a chatbot at Buriram Rajabhat covering eight service areas. These are built on fixed intents, so their coverage is limited to the questions and flows their developers wrote in advance. None of the ones we found answers from the university's own regulation documents, and none completes a request and reports its state inside the same conversation. We do not claim that nobody has built conversational university services. Our claim is about the kind of system: it retrieves from the university's own documents, it covers several services, it acts through tools, it cites its source and it tracks state.

1.2 How We Established These Problems

Method. On 5 and 6 October 2026 we reviewed the public websites of 17 universities, 12 in Thailand and 5 international. For each one we answered the same seven capability questions. C1: a student can request a document online. C2: the student can see its status. C3: a student can book a room online. C4: an assistant or chat channel exists. C5: the assistant answers from the university's own regulations. C6: the assistant completes a request in conversation. C7: these services are reachable from one place. We used public pages only and did not log in. We also reviewed published Thai research on university LINE chatbots.

Limits of this method. Anything behind a student login is invisible to us, so a university may offer more inside its portal than we can see. A cross means the capability is not offered to students, or that we found no public evidence of it after a targeted search of that university's own sites. Every row can be rechecked from the sources listed at the end of this section. We did not run a student survey, because the team has no way to reach a sample large enough to mean anything. The website review replaces it.

✓ capability present ~ partly present ✗ not offered, or no public evidence found

University	C1	C2	C3	C4	C5	C6	C7	Evidence and notes
Chulalongkorn (TH)	✓	✗	✓	✓	✗	✗	✗	Public request form, student number plus ID, QR payment. Documented process ends at payment and collection. Rooms booked in the CUNEX app. Registrar LINE account @regchula answers only.
Thammasat (TH)	✓	✗	✓	✓	✗	✗	✗	Requests on reg.tu.ac.th. Study rooms booked through QueQ, a separate product. Library LINE account @tulibrary for news and enquiries.
Mahidol (TH)	✓	✗	✓	✓	✗	✗	✗	Online proof-of-education system behind login, 1 to 5 working days then collect or post. Online syndicate room reservation, at the College of Management library. LINE account for registration queries.
Kasetsart (TH)	✓	✗	✓	✓	✗	✗	✗	Graduate request system for graduate students, available 24 hours, confirms by email only. Smart Study Room booking on a separate library system, at the Kamphaeng Saen campus library. Library enquiries by phone and LINE.
KMUTT (TH)	✓	✗	✗	✓	✗	✗	✗	Online document request form. Library states KM Rooms are walk-in only, so rooms cannot be booked online. Registrar LINE account @regiskmutt.
KMITL (TH)	✓	✗	✓	✓	✗	✗	✗	Digital transcript request behind login. Library room and area booking for students and staff. Library chat assistant "NongLifelong".
Assumption (TH)	✓	✗	✗	✗	✗	✗	✗	Online transcript and certification request. Ready in 5 working days, or about a month after finals, with no status view. Study rooms exist but no online booking found. No assistant or chat channel found.
Bangkok Univ. (TH)	~	✗	✗	✓	✗	✗	✗	"Online" means an email plus a photo of the bank transfer slip, so there is no form. No room booking on the library site. Student questions answered on LINE.
Srinakharinwirot (TH)	✗	✗	✗	✓	✗	✗	✗	No online request route found on public pages. A room booking system exists but is for teaching and administrative staff, not students. Library LINE account for notifications.
KMUTNB (TH)	✗	✗	✓	✓	✗	~	✗	Transcripts are requested in person at the counter, or by downloading a form, then collected with a slip. Smartroom booking across three campuses, plus a second room system, k-room. NN Bot on LINE sends notifications and information.
STOU (TH)	~	✗	✗	✓	✗	✗	✗	STOU SISA app covers records, results and enrolment with mobile payment, but we found no confirmed online document request route. An open distance university, so there are no campus study rooms to book. LINE account with a published service manual, answered by staff.
Buriram Rajabhat (TH)	✓	✗	✗	✓	✗	~	✗	Online request menu inside the student system, 2 to 3 working days then collect or post. No room booking found. LINE chatbot across eight service areas, built on fixed intents and published as research.
NUS (SG)	✓	✓	✓	✓	✗	~	~	Strongest in the survey. NUService Hub tracks request status in one platform. Rooms in LibCal and Chope@NUS, letters in myEduRec, so still several systems. okOSA takes a status letter request in chat, then takes up to 4 working days and reports nothing back in the chat.
Melbourne (AU)	✓	✓	✓	~	✗	✗	✗	Candidature requests tracked in my.unimelb. Rooms in DiBS and computers in BookIT, two separate booking systems. Its ask.unimelb service is being retired.
Toronto (CA)	✓	~	✓	✓	✗	✗	✗	Students generate some letters themselves in ACORN, but custom letters go through a separate eService with no status view. Rooms in LibCal. "Ask a Librarian" is a staffed chat shared across Ontario universities, not an assistant.
Manchester (UK)	✓	✗	✓	✓	✗	✗	✗	Online enquiry form for documents, fees and immigration, with no status view. Rooms in ResourceBooker, a separate system. "Reggie" answers quick registration questions.
Arizona State (US)	✓	✓	✓	✓	✗	✗	✗	Transcripts ordered through My ASU and fulfilled by Parchment, a third party, where the order status can be followed. Library rooms booked online. Sunny answers questions on housing, financial aid and advisors.

Present out of 17	**13**	**3**	**11**	**15**	**0**	**0**	**0**	Counting full ticks only. Partial marks: C1 two universities, C2 one, C4 one, C6 three, C7 one.

What the review shows. An online document request route is common, found at 13 of the 17 universities, so a missing service is not the problem. Request status visible to the student is rare, found at only 3 of 17 and at none of the 12 Thai universities. Some kind of assistant or chat channel exists at 15 of the 17, but every one we found answers questions, sends notifications or follows fixed intent flows, and none of them answers from the university's own rules (C5: 0 of 17) or completes a request and then reports its state in the same conversation (C6: 0 of 17). Nothing brings the services into one place (C7: 0 of 17). Room booking is also common and is handled by separate products such as LibCal, CUNEX and QueQ, which reinforces the first problem rather than solving it.

Sources, all checked 5 to 6 October 2026: *reg.chula.ac.th, reg.tu.ac.th, graduate.mahidol.ac.th, grad.ku.ac.th, regis.kmutt.ac.th, reg.kmitl.ac.th, registrar.au.edu, bulinks.bu.ac.th, nusit.nus.edu.sg, nus.libcal.com, nus.edu.sg/inside-nus, gradresearch.unimelb.edu.au, sis.utoronto.ca, studentsupport.manchester.ac.uk, ssi.asu.edu, springshare.com/libcal. Published research reviewed: a 2025 review of university chatbots in Springer, an IJECE paper on LINE-based graduate student services, and Thai journal papers on NN Bot at KMUTNB and a student service chatbot at Buriram Rajabhat.*

1.3 Project Objectives

1. Index the regulation corpus and answer at least 80 percent of a 50-question test set correctly, with a source citation on every answer.
2. Move the student's common tasks into one conversation: ask a policy question, read their own record, book an advisor appointment or a study room, search the library catalogue and renew a loan, and submit and track a service request.
3. Give every booking and every request a durable state and a visible history, and tell the student when the state changes without being asked, so that no student travels to campus to ask about progress.
4. Preserve conversation context across staff changes, across channels and across days, so that a student never restates a case the institution already holds. A staff member who takes over reads a summary and a draft reply.
5. Let staff, instructors and administrators do routine work by asking the same assistant: the queue, a bulk reminder, attendance and advising for instructors, and account and document jobs for administrators. The linked account sets the role, the role sets the tools, every write asks for a yes, and every write leaves an audit record.
6. Keep every component free to use, and every component we host open source. The language model runs on the OpenRouter free tier, and the build budget is zero.
7. Show that the system makes the task easier, by comparing the number of steps and the time a student needs to submit one request and check its status, before and after.

1.4 Target User Profile and Operational Environment

The environment is the student services function of the University: an office of three to six staff, instructors who also act as advisors and hold weekly office hours, a facilities team owning bookable rooms, a library, and a student body that already talks to each other through chat apps. The office is open on weekdays only, and the number of questions rises sharply around registration, fee deadlines and result days. Students are comfortable with chat apps and not with university software. They use a phone, often outside office hours, in English or Thai, and they will not read a manual. Staff are comfortable with office software and a shared inbox and are not developers. Instructors want to spend no time administering a calendar. The administrator is technical and owns accounts, roles and the deployment. A guest, such as an applicant, reads the public site and checks an application status, and a guest never uses the assistant.

1.5 System Context

Figure 1 in Appendix B shows the system. A student reaches it through the public site, the portal, or a chat channel. Staff, instructors and administrators sign in to the same React application and reach their own console. Chatwoot owns the three channels and the human handoff and calls the application by webhook. One supervisor reads the role of the signed-in account and routes each task to one of four agents: Knowledge, Records, Booking and Request. A memory store, keyed by account, carries facts across days and channels. A scheduler and worker send reminders,

watch notices and retries, and an ingestion worker indexes the documents that an administrator uploads. On LINE and Telegram the assistant sends plain text only, so the user types yes to agree and can still send a photo.

Retrieval-augmented generation (RAG) means the model answers only from text the system retrieves first. OpenID Connect (OIDC) is the protocol Keycloak uses to issue and verify login tokens. OpenRouter is a service that gives one API to many hosted language models, some of them free.

Section 2: Multi-Dimensional Feasibility Analysis

This section uses the four dimensions named in the assignment brief. The writing guide's six dimensions map into them: Resource Feasibility sits in 2.2, and Hardware Access and Risk Identification and Mitigation sit in 2.4.

2.1 Technical Feasibility

Stack, all free or open source. Python 3.12 with FastAPI and Uvicorn for the backend, because the agent ecosystem is strongest in Python and every agent turn waits on a model call. LangChain and LangGraph for the agents. The supervisor is one LangChain agent that calls each of the four sub-agents as a tool, and a role middleware filters the tools on every model call. We chose this over the langgraph-supervisor library because that library is no longer maintained and its documentation now recommends sub-agents as tools. OpenRouter free models for inference, behind one adapter. PostgreSQL 16 with pgvector, so one engine holds both transactional records and document vectors. Keycloak for identity, which removes all credential handling code. Self-hosted Chatwoot, which supplies the web widget, the LINE and Telegram connectors and human takeover in one service. React with Vite as a single application with Keycloak role-gated routes. Docker Compose with an ngrok tunnel for channel webhooks. pytest and Locust for testing.

Architecture realism. Docker Compose runs ten containers on one host: the React build, the API, one worker, the application database, Keycloak with its database, and Chatwoot with its web, worker, Redis and database containers. The model is the one outside service. The code we write is two processes, the API and the worker, built from one code base. There is no message broker of our own, no microservices and no Kubernetes. The multi-agent part is one supervisor inside the API process, so when an agent fails we debug one program instead of many. The worker handles reminders, retries, watches, nightly signals and document ingestion. We keep the feature list small, and the features that remain share code. Every bookable thing is one resource row with a slot table, and every trackable thing is one request row with a type and a state. Two pieces of code therefore cover every booking and every request type, and we write each one only once. The extended features follow the same rule. Each one is a tool on an existing agent plus a table, so it needs no new agent. Figure 2 in Appendix B shows how a message is routed and how a tool is chosen.

API access and rate limits. The model is hosted, so no laptop has to run it, and the build needs no graphics card. The cost is a quota. OpenRouter limits free models to 20 requests a minute. It also limits them to 50 requests a day for an account that has bought fewer than 10 credits, and to 1,000 requests a day for an account above that. These limits belong to the account, not to a key, so extra keys do not add capacity. One assistant turn uses several model calls, because the supervisor makes one call and each agent it uses makes at least one more. The evaluation in Section 4 needs 150 questions and messages, and it will be run more than once. We handle this in three ways. First, evaluation runs are cached, so a re-run of an unchanged test sends no new request. Second, the team decides before the evaluation starts whether to buy 10 credits, which is a one-time cost of 10 US dollars. Third, a free model can be withdrawn without notice, so all model calls go through one adapter file, and a change of model or provider changes settings and that one file, not the agents.

Production note. The free tier is enough for a prototype with no budget and not for a real service at student volume. A real deployment moves to a paid tier, and it needs a provider agreement for student data, because a hosted model receives the text of every conversation. Our data is synthetic, so the prototype has no such exposure. The database runs in Docker Compose for development, tests and the demonstration. A managed host such as Neon is the production option, and the team has not chosen it yet.

Skills and gaps. Some of us have used parts of this stack before, in coursework, so we are not starting from zero on FastAPI, PostgreSQL, LangChain, RAG, Keycloak and Chatwoot. Whoever already knows a tool will share it with the rest of the team while we build, and the first two weeks leave time for that. A one-day test of Chatwoot (11

checks, 9 passed) showed that a chat stays pending when a bot does not answer, and that a failing bot address opens the chat for a person. Human takeover itself is still a design, which we build and test in Project I. Three real gaps remain. Tool calling with a free hosted model is the first one. Free models follow tool instructions less reliably than paid ones, so we test two or three of them in Week 2 and pick one from the results, before we write any agent. Measuring answer quality is new to us, so we write the test set before the agents and have two people score the answers separately. Thai search and load testing are also new. We choose a multilingual embedding model by testing Thai questions from Week 2, and we use Locust for load testing, which takes about one day to learn.

2.2 Operational and Resource Feasibility

Students already answer questions through chat on a phone, so students do not have to learn a new habit, and LINE is a channel this group opens daily. The adoption risk is with the staff, not the students. An assistant that answers questions takes away work they own today, and a console is a new tool in a workflow that runs on a shared inbox. Our design answers both. The assistant never decides on a payment, a request or a booking for a person. Every write asks for a yes and leaves an audit record, and the assistant passes the conversation to a person when the student asks or when it is not sure. The staff console shows one queue that looks like the list the office already keeps. Staff can also ask the assistant, for example for the queue in one sentence, and they then say yes to an approval. Staff training is one page covering four actions: take over a conversation, change a request state, ask the assistant to record a payment after checking a slip, and release the conversation. Students need no training.

Tooling, hosting and budget

Build budget: zero. Docker images for all services, the OpenRouter free tier for inference, the ngrok free tier for channel webhooks, and the GitHub free plan. The one optional purchase is 10 OpenRouter credits for the evaluation window, described in 2.1. Memory is no longer the main limit, because the model does not run on our laptops. One laptop runs the full container stack as the demonstration machine, and the other three run the application and PostgreSQL for development.

Team allocation, split by layer rather than by feature. Track 1 owns infrastructure and identity and publishes the database schema and token claims. Track 2 owns the English and Thai corpus, indexing, the test set and all measurement, and publishes the retrieval function signature. Track 3 owns the supervisor, the role middleware, the four agents and every tool, and publishes the chat endpoint. Track 4 owns the React application, the public pages, the portal, the role consoles and the three channels. The team has not assigned the four tracks to people yet, and it does so before the build starts. If four people edit the same agent file we get merge conflicts, and if each person works behind an agreed interface we do not. So we agree the four interfaces in W1 and each track builds against a simple fake version of the others until the real one is ready.

2.3 Time Feasibility

Status on 17 October 2026. The problem statement, the 17-university survey, the architecture and data diagrams, and a clickable prototype are done. The prototype has 315 frames in 15 stories across four parts (student, staff, instructor and administrator), and a link check found no broken link. The midterm needs no code. The code follows the plan below.

Phase	Delivered	Done means
W1–W2	Compose stack, schema, Keycloak realm, OpenRouter adapter, interfaces agreed. Test set written and free models compared before any agent code.	A user signs in, and the model is chosen on measured evidence.
W3–W5	Core. Corpus indexed. Knowledge, Records, Booking and Request agents with the supervisor, the role middleware and the audit writer. Slot locking, the four-state request lifecycle, library search and renewal, memory. Portal views. FR-01 to FR-19 except the channels.	The core journey works end to end, and the concurrent booking test shows zero double-bookings.
W6–W8	Channels and roles. Chatwoot with Telegram then LINE, handoff with summary and draft, the staff, instructor and administrator consoles with the assistant, the payment record flow, request files, and the worker with reminders, watches and retries.	One conversation runs across three channels, and each role completes its main task by chat.
W9	Extended features and measurement. Attendance, advising brief, signals and study plans, bulk reminders, account jobs, document upload, guest application. Access-control negative suite, load test, final scoring, usability test with five students.	Every target in Section 4.3 is measured, and every requirement has a test result.

W10	**Buffer. No new features.** Defect fixing, documentation, demonstration rehearsal.	We have room to fall behind without cutting the main features.

The scope is large for four people. Section 4.2 lists 14 extended requirements (FR-20 to FR-33) beside the 19 core ones, and we accept that size because each one is a tool and a table on an agent that already exists. We therefore start the extended features only after the core passes its tests. Two of the ten weeks add no new features. W2 is for measuring, and W10 is for fixing bugs. If we fall behind, we have already decided what to drop first. The cut order is study plans and advisee signals, then bulk reminders and account creation from a file, then LINE, then the study-room resource type. The core in FR-01 to FR-19 is never cut. Deferred to Project II: cloud deployment with a paid inference provider, direct channel calls from the backend, course add and drop with prerequisite and timetable-clash checking, payment processing, integration with a real student information system, and full-text retrieval inside library holdings.

2.4 Data Feasibility and Key Risks

We use no real university data, and we do not need any. We have no partner university, and asking for real student records would raise a privacy problem we cannot handle as students. Every data set is constructed: the regulation corpus is authored from publicly published university handbooks in aligned English and Thai versions; student records, charges, loans and bookable resources are generated with synthetic names and identifiers; and the evaluation sets are 50 graded test questions and 100 labelled routing messages, frozen before agent development. All of it is committed to the repository, so every evaluation run is reproducible.

Privacy and security. We build the system as if the data were real, because getting access control right is the part that matters most. Every agent tool takes the subject identifier from the verified login token and never from the message text. The tool set of a role is unreachable from a session of a lower role. Every write asks for a yes, is capped, and writes an audit record. Personal identifiers are excluded from logs, and all browser traffic uses TLS. A payment slip is stored privately, and no agent reads it. A hosted model sees conversation text, so only synthetic text goes to it in Project I.

Technical and schedule risks, with mitigation

ID	Risk	L / I	Mitigation
R1	The free tier rate-limits us (20 requests a minute, 50 a day under 10 credits), or a free model is withdrawn, or free models are unreliable at tool calling.	High / Medium	We test tool calling on two or three free models in Week 2 before we write any agent. All calls go through one adapter, so a model change needs no agent change. Evaluation runs are cached. The team decides before the evaluation whether to buy 10 credits, which raises the daily limit to 1,000. When the model is rate limited, the assistant says so and offers a human within 10 seconds.
R2	Prompt injection: text in a document or a message instructs the agent to act or to reveal another user's data.	Medium / High	Tools take the subject from the token, never from text. The role middleware gives each role its own tool list, and no tool accepts free-form queries. Every write needs a yes. A negative suite attempts cross-account and cross-role access on every tool in continuous integration.
R3	Scope creep, because a university offers an endless supply of plausible additions.	High / High	Section 6 freezes the list. Adding something means removing something of the same size, and we only accept it if it reuses the booking code, the request code or the read code. In October the team added library search, loan renewal, the payment record flow, the assistant for staff, instructors and administrators, and the extended features, and removed the fixed slip reader.
R4	Integration failure between four parallel tracks, or a campus network that blocks the tunnel or the model service.	Medium / High	We agree four interfaces in W1 and each track builds against a simple fake version of the others. We test network access in W2, not on demo day, and our backup is a shared phone hotspot.
R5	Schedule slip or uneven contribution across a four-person team.	Medium / High	Each track carries a weekly milestone with a binary done condition, and the four tracks demonstrate against the frozen interfaces at a weekly integration checkpoint. One feature works end to end by W4, W10 adds no new features, and we agree in advance what to drop first instead of deciding under pressure.

R6	Thai quality falls far below English, or the agent gives a confident answer the documents do not support.	Medium / Medium	Thai questions are in the test set from W2, so we see the gap early. If Thai answers are weak, the agent shows the Thai passage it found with its source instead of writing a summary. Every answer carries a source, and if nothing scores above the similarity threshold the agent refuses instead of making something up.
R7	LINE message quota. Chatwoot sends each LINE message as a push message, and push messages count against the monthly quota of the LINE account plan. Reply messages do not.	Medium / Low	Demonstration traffic is small. The team decides whether the backend replies on LINE directly, which uses the free reply route, or stays with Chatwoot. Telegram has no such quota.
R8	The scope is too large for ten weeks: 33 functional requirements, 44 data entities and four consoles for four people.	High / High	The core (FR-01 to FR-19) is built and tested first. The extended features start in W6 and each reuses an existing agent, so none needs a new agent. The cut order in 2.3 is fixed in advance. Each track reports a weekly binary milestone, and the team cuts at the first missed milestone instead of in W10.

Section 3: Feasibility Recommendation

PROCEED WITH ADJUSTMENTS

The project is feasible for a four-person team within Project I, with the adjustments below. The code is one API and one worker built from one code base, and the supporting services are common and well documented. The main technical risks are the free model tier, with its rate limit and the tool-calling quality of free models, and the size of the scope. We handle it with a model test in Week 2, one adapter, cached evaluation runs and an optional purchase of 10 credits. Every tool is free or open source and the model is hosted, so the build budget is zero. The feature list is large, so we split it into 19 core and 14 extended requirements, and the features share code: one piece of code books any slot and one tracks any request, and each extended feature is a tool on an existing agent. Four changes keep the schedule honest: we choose the model by testing it in Week 2 instead of assuming, we build the core before any extended feature, we fix the cut order in advance, and we support English and Thai only and measure Thai from the start. Events, maintenance reports, library borrowing and the other items in Section 6.2 move to Project II. With those adjustments the project is ready to move to requirements and system architecture.

Section 4: Stakeholders and Requirements Specification

4.1 Stakeholder Register

Class	Stakeholder	Interest and influence
Primary users	Enrolled students; student services staff	Students want an immediate correct answer and a task finished without visiting a counter. Staff want fewer repeat questions, any case with its full history, and routine updates by one sentence. Both have high influence, because the system has no value if either group does not use it.
Secondary users	Instructors who act as advisors; facility and library staff; head of student services	Instructors and facility owners want one booking per published slot and own the resource data. Instructors also ask the assistant for their schedule, their advisees and their class attendance. The head of office wants volume and turnaround visibility and sponsors the deployment decision. Medium influence.
Administrators	IT administrator	Manages accounts and roles, runs the container stack, keeps the corpus current, and uses the assistant for account jobs and system questions. High influence, because they control deployment, access and account provisioning.
Platform	IT team	Runs the platform and the Chatwoot admin screen, sets up the inboxes, the channels and the bot, and signs in to Chatwoot with its own login. The IT team is not a role in the application. Medium influence, because the channels depend on it.

External actors	OpenRouter; LINE Messaging API; Telegram Bot API; guests and applicants	The model service generates every reply, so it is high influence and a single point of failure. The channel APIs are low influence, because the web widget works without them. Guests read the public site and never use the assistant.

4.2 Functional Requirements

A bookable resource in Project I is an advisor appointment or a study room. A service request is a document request or a library loan renewal. One requirement therefore covers every instance of its type, which is how the code is written. The test method is stated in brackets after each requirement. FR-01 to FR-19 are the core. FR-20 to FR-33 are the extended features, which the team builds after the core and which the cut order in Section 2.3 names.

1. **FR-01** The system shall authenticate every user through Keycloak using OpenID Connect and shall reject with HTTP 401 any request to a protected endpoint, including the chat endpoint, that carries no valid access token. A guest therefore cannot use the assistant. (Tested with an absent, an expired and a valid token.)
2. **FR-02** The system shall assign each account exactly one of the roles student, instructor, staff or administrator and shall return HTTP 403 when an account calls an endpoint its role does not permit. (Every endpoint called once per role.)
3. **FR-03** The system shall route each task in a message to one of the specialist agents Knowledge, Records, Booking and Request, and shall store the chosen route with the message identifier. (Routing log read after the 100-message labelled set runs.)
4. **FR-04** The system shall answer a policy question only from passages retrieved from the indexed corpus and shall display the source document title and section with every answer. (All 50 test answers inspected. Zero without a citation.)
5. **FR-05** The system shall refuse to answer and shall offer a human handoff when no retrieved passage scores above the configured similarity threshold. (10 questions with no corpus coverage. All 10 refuse.)
6. **FR-06** The system shall accept messages in English and in Thai and shall reply in the language of the incoming message. (20 English and 20 Thai questions. Language matches in all 40.)
7. **FR-07** The system shall return the signed-in student's own timetable, registration status, outstanding fee balance and library loans, and shall refuse any student request that names another student. (30 other-student retrieval attempts. 30 refusals expected.)
8. **FR-08** The system shall list the available slots of an advisor appointment or a study room for a given date range. (Compared against the database for 10 ranges across both resource types.)
9. **FR-09** The system shall reserve a selected slot inside a single database transaction and shall reject any second reservation of a slot already taken. (200-request concurrent test on one slot, run against the booking tool. Exactly one success.)
10. **FR-10** The system shall allow a student to cancel or reschedule only their own reservation, and only before the cut-off configured for that resource type. (Tested before the cut-off, after it, and on another student's reservation.)
11. **FR-11** The system shall accept a service request of a configured type, shall hold it in exactly one of the states Submitted, Under review, Ready or Collected, and shall record a timestamped history entry for every state change. (One request of each type moved through all four states, history read back.)
12. **FR-12** The system shall notify the requesting student on their originating channel within 60 seconds of a change to the state of their reservation or request. (Delivery measured on web, LINE and Telegram.)
13. **FR-13** The system shall hand a conversation to a human staff member on student request or on low routing confidence, shall let the staff member reply from the staff console, and shall generate no further automatic reply until the staff member releases it. (Handoff triggered, reply sent from the console, silence confirmed for 10 further messages.)
14. **FR-14** The system shall exchange messages on the web widget, LINE and Telegram through Chatwoot, shall present one continuous conversation history per student across all three channels, and shall send plain text only on LINE and Telegram. (Conversation started on web, continued on LINE and Telegram, one history read back, no button or card in any channel reply.)

15. **FR-15** The system shall ask the user to confirm with a yes before it writes any booking, request or record, and shall write an audit record of actor, action, target and timestamp for every write an agent performs, for every read of personal data, and for every blocked call. (One booking and one request submitted, the confirmation step observed, then both audit rows read. One personal read and one blocked call, then both rows read.)
16. **FR-16** The system shall search the seeded library catalogue by title, author or subject through a read-only query, and shall return only catalogue data. (20 searches compared with the database.)
17. **FR-17** The system shall renew a library loan for the signed-in student only when the renewal rules allow it, after a yes, and shall write the new due date and an audit record. (One allowed renewal, and one refusal for each rule, read back.)
18. **FR-18** The system shall accept a payment slip image from a student as a normal chat message, shall not read the image, and shall pass the chat to staff. When a staff member asks the assistant in a separate chat to record the payment, the system shall ask for a yes and then write a payment record and an audit record in one transaction. (Slip sent on LINE, staff reply by hand, record written after the yes, balance read back, image never passed to a model.)
19. **FR-19** The system shall give each role its own tool set and shall let staff, instructors and administrators use the assistant for the actions that role permits. (10 attempts to call a tool of another role. 10 refusals expected.)
20. **FR-20** The system shall save facts that a user states or settles in a chat, shall key them to the account and not to the channel account, and shall load them in the next conversation on any channel. (10 facts saved on LINE and read back on the web the next day.)
21. **FR-21** The system shall send a reminder to the student 24 hours before an appointment, and shall message a student who watches a request when its state changes, on the channel where the student asked. A reminder arrives within 5 minutes of its time, and a watch message within the 60 seconds of FR-12. (Seeded appointments and watched requests. Every message arrives in time.)
22. **FR-22** The system shall keep a question when a tool it needs fails, shall tell the sender that the answer follows, shall retry every 5 minutes, and shall send the answer when the tool works. (Tool made to fail, then restored. The answer arrives once, with no duplicate.)
23. **FR-23** The system shall give the staff member who takes over a conversation a summary of the chat and a draft reply, and shall send nothing until the staff member sends it. (10 handoffs. A summary and a draft exist in all 10, and no draft is sent without the staff member.)
24. **FR-24** The system shall record a question as unanswered when no passage scores above the threshold, and shall record a source conflict when the cited passages disagree, and an administrator shall read both lists. (10 questions with no coverage and 2 seeded conflicting documents. All 12 recorded.)
25. **FR-25** The system shall let an administrator upload a document, shall split, embed and index it in the background, and shall answer from it once indexing ends. (One document uploaded. A question on its content is answered with a citation to it.)
26. **FR-26** The system shall accept a file or photo that a student sends for an open request, shall store it privately, and shall attach it to the request for staff to read, and no image shall go to a model. (One file sent on LINE. Attached to the right request, and absent from every model call in the log.)
27. **FR-27** The system shall tell an instructor, for the instructor's own courses only, who missed a class and who is near the absence limit, and shall send an absence warning to chosen students after a yes. (Four questions on a seeded course. A question about another instructor's course is refused.)
28. **FR-28** The system shall prepare an advising brief for each appointment, shall flag advisees with rule-based signals, and shall store a study plan that the advisor approves after a yes. (One brief per seeded appointment. Seeded advisees flagged by the expected rules. One plan approved.)
29. **FR-29** The system shall let a staff member ask for a bulk reminder to a named group, shall show the group size and a sample first, and shall send only after a yes. (One reminder to a group of 20. The count matches, and nothing is sent after a no.)
30. **FR-30** The system shall let an administrator manage accounts and roles, create accounts from a file after a yes, and read the system status, the audit log and the blocked calls. (A file of 10 accounts. Ten accounts created, 10

audit rows read.)

31. **FR-31** The system shall compute grade sums, what-if grades and graduation plans in tool code and not in the model, and the model shall only put the result into words. (20 calculations compared with a hand count. All 20 match.)
32. **FR-32** The system shall let a guest, without a login, read the public pages, apply for a program, check an application with its reference number and send a contact message. (One application sent, the reference returned, the status read back.)
33. **FR-33** The system shall record the route, the tool steps and the model usage of every reply, and shall let a user rate a reply as helpful or not. (100 replies. The steps and the usage are present for all 100. One rating stored.)

4.3 Non-Functional Requirements

Performance targets are set for a hosted free model, which is faster than the local model assumed in version 2, so they are tighter. The system streams its reply, so first visible token and completion time are both stated. The targets are proposals until the Week 2 model test measures them.

1. **NFR-01 Performance.** 95th-percentile time to first visible token shall be 3 seconds or less, and to complete reply 10 seconds or less for a retrieval answer and 6 seconds or less for a records lookup, over 100 requests. The system shall sustain 10 concurrent conversations with no failed request. The load test replaces the model call with a recorded response, because the free tier allows only 20 requests a minute.
2. **NFR-02 Security.** Authentication shall use OpenID Connect with RS256-signed tokens expiring within 15 minutes. All browser traffic shall use TLS 1.2 or higher. The application shall store no password. Of 30 cross-account access attempts and 10 attempts to reach a tool of a higher role, 0 shall succeed.
3. **NFR-03 Usability.** A first-time student shall complete "book a study room" in 90 seconds or less and in 4 messages or fewer, with no written instructions. Of 5 test users, at least 4 shall succeed.
4. **NFR-04 Availability and reliability.** The stack shall start from one docker compose up within 5 minutes on a clean machine. A container restart shall lose no reservation and no submitted request. If the model is rate limited or unreachable, the system shall return a service message within 10 seconds and offer a human, rather than waiting indefinitely.
5. **NFR-05 Accuracy.** Routing accuracy shall be 90 percent or higher on a separate test set of 100 labelled messages. At least 80 percent of the 50 test questions shall be answered correctly with a correct citation, scored by two independent raters and reported separately for English and Thai.

Section 5: User Stories and Use Cases

5.1 User Stories

1. As a student, I want to ask about a regulation in chat and receive the answer with the rule it came from, so that I do not have to search PDF handbooks or rely on what a senior told me.
2. As a student, I want to book an advisor meeting or a study room inside the same conversation, so that I do not queue for a slot someone else already took.
3. As a student, I want to submit a document request or renew a library loan and check its state from LINE, Telegram or the website at any hour, so that I do not travel to campus only to ask whether it is ready.
4. As a student, I want to send my payment slip in the chat I already use, so that I do not email it to an office.
5. As a student, I want to continue tomorrow a chat I started today, on any channel, and to be told when my request changes state, so that I never repeat myself or chase the office.
6. As a staff member, I want to take over a conversation and see everything the student already explained, so that a student whose case I inherit does not have to start again.
7. As a staff member, I want to tell the assistant "Wai Yan Lynn, 67111411 paid this semester's fee, record it" after I checked the slip, so that the record changes without me opening a second system.
8. As an instructor, I want to ask who missed a class and who is near the absence limit, and then ask the assistant to message those students after I say yes, so that a warning goes out the same day.

9. As an administrator, I want to ask the assistant for a system answer or an account job, and to see an audit record of every change, so that I can account for each write.
10. As an administrator, I want to see the questions the assistant could not answer and the documents that disagree, so that I know which document to fix.

5.2 Detailed Use Cases

UC-01: Reserve a Bookable Resource

ACTORS

Primary: Student. Secondary: the resource owner, who is an instructor or the library. Supporting: Keycloak, PostgreSQL, OpenRouter.

PRECONDITIONS

The student holds an administrator-provisioned account and is signed in through Keycloak. The resource owner has published at least one availability window. The application, the database and the model service are reachable.

POSTCONDITIONS

Success: exactly one reservation row exists for that slot, owned by this student, the slot is no longer available, and an audit record and a confirmation message exist. Failure: no reservation row is created and the slot keeps its previous state.

MAIN FLOW

1. The student writes in the chat that they want a study room on Thursday afternoon.
2. The supervisor classifies the message as a booking intent and routes it to the Booking agent.
3. The Booking agent reads the student identifier from the verified access token.
4. The agent identifies the resource type and queries free slots for the requested date range.
5. The agent lists up to five options as plain text with resource name, day and time.
6. The student replies with one option, and the agent repeats the choice and asks for a yes.
7. The student types yes.
8. The agent opens a transaction, locks the slot row, confirms it is still free, writes the reservation and commits.
9. The agent writes an audit record of actor, action, target and timestamp.
10. The system confirms the resource, day, time and location, and states the cancellation cut-off.

ALTERNATIVE AND EXCEPTION FLOWS

A1. Slot taken during selection (step 8). The lock shows the slot is no longer free. The transaction rolls back, the agent reports the conflict, re-queries availability and returns to step 5. No reservation is written.
A2. No availability in the range (step 4). The agent reports that no slot is free, offers the next available period and returns to step 4 with the new range.
A3. Ambiguous resource or date (step 4). For "book a room" the agent asks which resource type is meant. For "next week" near a month boundary the agent states the exact date range it will use and asks for confirmation before querying.
A4. The student answers no (step 7). The agent writes nothing and offers other slots.
E1. Session expired (step 3). The token is invalid. The system does not guess an identity. It asks the student to sign in again and discards the pending action.
E2. Overlapping reservation already held (step 8). The agent refuses the second reservation, shows the existing one and offers to reschedule it.
E3. Model rate limited or unreachable, or malformed tool arguments (any step). The adapter retries and then posts a plain-text message within 10 seconds that says the model is rate limited and offers a human handoff. For malformed arguments the validation layer rejects the call and retries once, then escalates to a human rather than guessing.
E4. Database unavailable (step 8). The transaction rolls back. The system reports that the booking did not complete and tells the student to try again. No partial state is written.

UC-02: Record a Payment After a Slip

ACTORS

Primary: Staff member. Secondary: the student who sent the slip. Supporting: Chatwoot, Keycloak, PostgreSQL, OpenRouter.

PRECONDITIONS

The student has an open charge. The staff member is signed in through Keycloak with the role staff. The student sent the slip as a photo in a chat, and a staff member checked it in a normal chat and replied to the student by hand.

POSTCONDITIONS

Success: one payment record and one audit record exist, the charge is Paid, and the balance of the student changed. Failure: no record is written and the charge stays Open.

MAIN FLOW

1. The student sends the slip photo on LINE. The agent does not read the image, tells the student that staff will reply, and leaves the chat pending for staff.
2. A staff member opens the chat in the staff console, checks the slip against the bank statement and replies to the student by hand.
3. In a separate assistant chat, the staff member writes that student 67111411 paid this semester's fee and asks to record it.
4. The role middleware gives the supervisor the staff tool set, and the supervisor routes the message to the Request agent.
5. The Request agent finds the student by identifier and reads the open charge.
6. The assistant states the student, the charge and the amount, and asks for a yes.
7. The staff member types yes.
8. The Request agent writes the payment record and an audit record in one transaction, and the charge becomes Paid.
9. The assistant confirms the new balance to the staff member.

ALTERNATIVE AND EXCEPTION FLOWS

A1. The identifier matches no student, or two students (step 5). The assistant asks the staff member for the full identifier and writes nothing.
A2. The student has no open charge (step 5). The assistant says so and shows the last payment record.
A3. The staff member answers no (step 7). The assistant writes nothing.
A4. A payment record already exists for this charge (step 8). The transaction rejects the second record and the assistant shows the first one.
A5. The staff member reverses a record. The assistant asks for a reason and a yes, then returns the charge to Open and writes an audit record.
E1. The account is not staff (step 4). The role middleware removes the tool, the assistant refuses, and the refused attempt is written to the audit log.
E2. Model rate limited or database unavailable (any step). The assistant reports the failure, and no partial state is written.

Section 6: Controlled Scope Statement

6.1 In-Scope: Project I Deliverables

1. A public university website with program, admission, announcement and contact pages, plus an authenticated student portal and role consoles for staff, instructors and administrators, delivered as one React application with Keycloak role-gated routes. Guests read the public site and apply, and never use the assistant.
2. Authentication and role-based access through Keycloak for student, instructor, staff and administrator, with accounts provisioned by the administrator.
3. One supervisor that calls four agents as tools: Knowledge (answers from the English and Thai corpus with a citation), Records (read-only data), Booking (reserve, reschedule and cancel an advisor appointment or a study room, with transactional slot locking), and Request (submit and track a document request through four states, renew a library loan, and write a payment record). A role middleware gives each role its own tool set.
4. Chat for staff, instructors and administrators. They ask for a schedule, advisees, the queue, a brief, a booking change or a system answer. Every write asks for a yes and writes an audit record.

5. Library catalogue search through a read-only query on seeded data, and library loan renewal.
6. The payment slip flow: the student sends the slip, staff check it in a normal chat, and staff ask the assistant to record the payment after a yes. The agent never reads the slip and never decides on a payment.
7. Human handoff through Chatwoot, with a summary and a draft reply for the staff member, where the agent stops replying while a staff member is handling the conversation, across three channels sharing one conversation history: web widget, LINE and Telegram. LINE and Telegram carry plain text only.
8. Memory across days and channels, keyed to the account. A worker that sends reminders and watch messages, retries a failed question every 5 minutes, and indexes uploaded documents.
9. Extended features for the roles: attendance questions and absence warnings, the advising brief, advisee signals and study plans, bulk reminders, account creation from a file, the lists of unanswered questions and source conflicts, grade and graduation planning computed in code, request files, and the guest application with its status.
10. A record of the route, the tool steps and the model usage of every reply, with a helpful rating.
11. Inference through the OpenRouter free tier behind one adapter file, with the model chosen by testing in Week 2.
12. An evaluation harness of 50 test questions in English and Thai, 100 labelled routing messages and the access-control negative test suite, plus a Docker Compose deployment that starts the full stack with one command.

6.2 Out-of-Scope: Deferred to Project II or Backlog

1. Library borrowing, returning and holds, event registration, maintenance reporting, and every bookable resource other than advisor appointments and study rooms. These reuse the same two pieces of code and are deferred so that Project I stays small.
2. Payment processing and any reading of a payment slip by a model. The system records a payment after a human checks it. It never moves money.
3. Integration with a real student information system, grade entry, result publication, and automated admission or scholarship decisions.
4. Course add and drop execution, and prerequisite or timetable-clash checking, which would need a scheduling engine the project does not build.
5. Full-text retrieval inside library holdings, which would need licensed content we do not hold.
6. Any channel beyond web, LINE and Telegram, and direct calls to the LINE and Telegram APIs from the backend. Native mobile applications. Voice input. Languages beyond English and Thai.
7. Cloud deployment with a public domain, managed TLS and a paid inference provider. Model training or fine-tuning, learning-management functions, analytics dashboards and multi-institution scaling.

6.3 Constraints, Assumptions and Limitations

Constraints. Ten working weeks for a four-person team, with W10 holding no new functionality. A build budget of zero, with one optional purchase of 10 OpenRouter credits. Four laptops. Docker Compose on a single host, with the model hosted by OpenRouter, so there is no orchestration platform and no horizontal scaling. The free tier limit of 20 requests a minute and 50 requests a day applies until credits are bought.

Assumptions. A free hosted model is reliable enough at tool calling for this design. We check this with the model test in Week 2 instead of assuming it. The free model we choose stays available during the build. Documents we write from public university handbooks are close enough to a real university's documents in length, structure and vagueness. Students use a chat interface without training. Resource owners will publish availability digitally. The interfaces frozen in W1 remain stable.

Limitations. The prototype needs an internet connection to reach OpenRouter, and the channel webhooks need an ngrok tunnel, so the demonstration depends on one laptop and a working network. Response times depend on the free tier and can change from day to day. All records are made up, so we show that the system works, but not how it copes with messy real data such as duplicate or incomplete records. Thai quality is expected to be lower than English quality, and the report states the measured gap rather than hiding it. We test usability with five users,

which is enough to find the big problems but is too small a group to be representative. We write the test set ourselves, so two of us score the answers separately. That reduces our own bias but does not remove it.

Appendix A: Changes From Version 2

Area	Version 2 (2 October 2026)	Version 3 (17 October 2026)
Model	Open-weight model on team hardware through Ollama. No quota. Memory table, dedicated model host, LAN and Tailscale plan.	OpenRouter free tier behind one adapter. Real limits stated: 20 requests a minute, 50 a day under 10 credits, 1,000 a day above. Hardware analysis removed.
Risk R1	A local model is unreliable at tool calling.	The free tier rate-limits us, or a free model is withdrawn. New risk R7 for the LINE message quota.
Agents	LangGraph with langgraph-supervisor.	One LangChain agent calls four sub-agents as tools, with a role middleware. The old library is no longer maintained.
Roles	Student, staff, advisor, administrator. Staff-side agent deferred to Project II.	Student, instructor, staff, administrator. Staff, instructors and administrators use the assistant in Project I.
Assistants in the survey	"Eight of the 17" run an assistant.	Fifteen of 17 offer an assistant or a chat channel (C4). None answers from the university's rules or completes a request.
Request states	Submitted, under review, ready, closed.	Submitted, Under review, Ready, Collected.
Scope	Library catalogue and payment slip reading out of scope.	Library search, loan renewal and the payment record flow in scope. The fixed slip reader is dropped.
Channel messages	Not stated.	Plain text only on LINE and Telegram. The user types yes. No buttons and no cards.
Requirements	15 functional, 5 non-functional. Performance 4 s and 15 s.	33 functional (19 core and 14 extended), 5 non-functional. Performance tightened to 3 s and 10 s, and stated as a proposal until measured. FR-16 to FR-33 are new.
Extended features	Not in the report.	Memory, reminders and watches, retry, handoff summary, unanswered questions, document upload, request files, attendance, advising, bulk reminders, account jobs, planning in code, guest application and the reply record. All are Project I scope, built after the core.
Deployment	One application process, one database, three services, four laptops.	Ten containers on one host, with a worker. The IT team is a new stakeholder.
Use cases	UC-01 only.	UC-01 with a confirm step, and a new UC-02 for the payment record.
Objectives	Model on team hardware. Book any campus resource.	Hosted free model. Advisor appointments and study rooms. New objective for staff, instructor and administrator chat.
Institution	A fictional bilingual university.	Thales University, a fictional university.

Appendix B: Diagrams

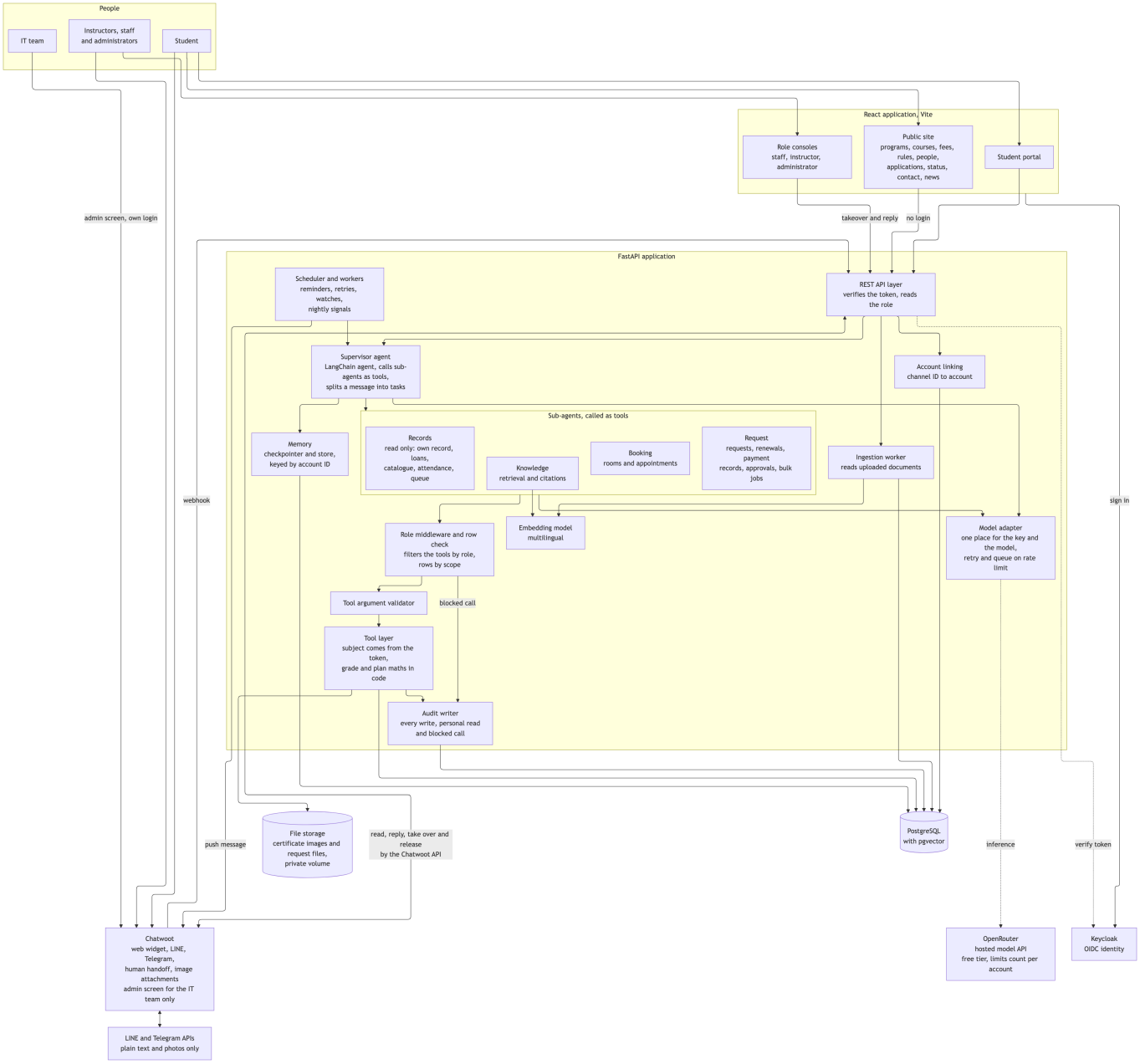


Figure 1. System architecture (design/diagrams/01_architecture).

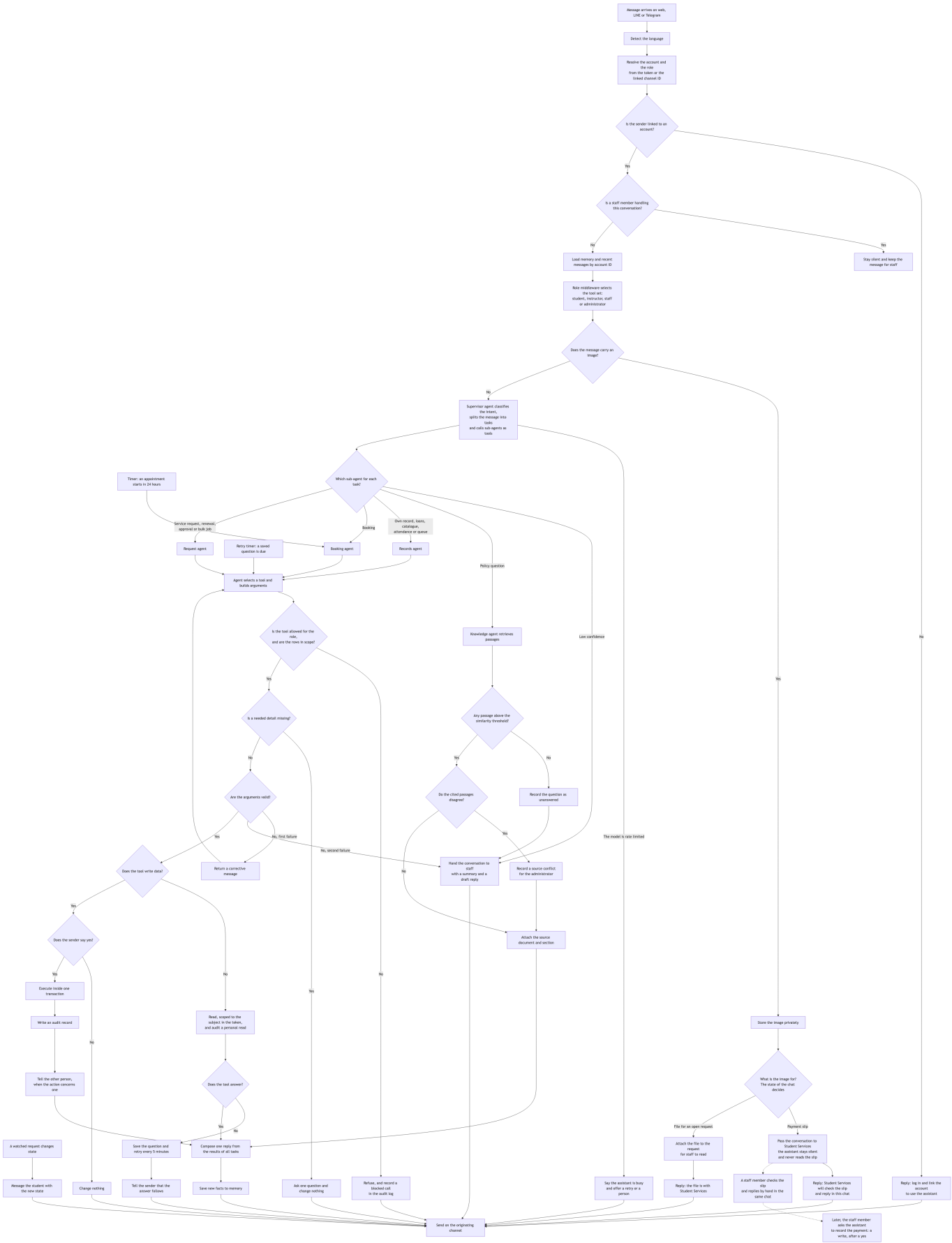


Figure 2. Supervisor routing and tool selection (design/diagrams/03_supervisor).